OBJECT ORIENTED ANALYSIS AND DESIGN (CT 651)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 1: Object Oriented Fundamentals

Syllabus Hours: 10 hrs | Marks Weight: 18 marks

CHAPTER 1 SYLLABUS TOPICS

- 1.1 Introduction
- 1.2 Object Oriented Analysis and Design
- 1.3 Defining Models
- 1.4 Case Study
- 1.5 Requirement Process
- 1.6 Use Cases
- 1.7 Object Oriented Development Cycle
- 1.8 Overview of the Unified Modeling Language: UML Fundamentals and Notations

SECTION 1: OOP, OOA, OOD and Object Oriented Development Cycle

Exam Ref: 2082 Kartik [3+5], 2081 Chaitra [2+5], 2081 Ashwin [5], 2080 Ashwin [5+3], 2079 Ashwin [4+4], 2079 Chaitra, 2078 Chaitra, 2077 Chaitra [4+6]

Q1. How are OOP, OOD and OOA related? Explain object-oriented development cycle in detail. [3+5] (2082 Kartik)

Answer:

OOP, OOA, and OOD — Relationship

Object-Oriented Programming (OOP), Object-Oriented Analysis (OOA), and Object-Oriented Design (OOD) are three complementary phases that together form the complete Object-Oriented Software Development process.

Term	Definition	Focus
OOA (Analysis)	Process of examining requirements and identifying real-world objects, their attributes, and relationships in the problem domain.	WHAT the system should do
OOD (Design)	Process of translating the analysis model into a solution — defining software objects, classes, and their interactions.	HOW the system will be built
OOP (Programming)	Process of implementing the design using an OO programming language (Java, C++, Python).	Actual CODING of the system

Relationship: OOA → OOD → OOP form a pipeline. OOA produces a conceptual model of the problem. OOD refines it into a blueprint. OOP realizes it as executable code. All three share the same OO concepts: classes, objects, inheritance, encapsulation, and polymorphism — creating seamless transition between phases.

Object-Oriented Development Cycle

The OO Development Cycle is iterative and incremental. Each cycle produces a partial but working system. It consists of the following phases:

Phase	Activity	Output
1. Planning	Define project scope, schedule, feasibility, and resources.	Project plan, feasibility report
2. OO Analysis (OOA)	Gather requirements, identify use cases, actors, conceptual classes, and domain model.	Use case model, domain model, SSD

3. OO Design (OOD)	Define design classes, collaboration diagrams, interaction diagrams, GRASP patterns, class diagrams.	Design class diagram, interaction diagrams
4. Implementation (OOP)	Write code from design classes, create method definitions, map design to code in OO language.	Executable source code
5. Testing	Unit test, integration test, system test, acceptance test.	Test reports, bug fixes
6. Deployment	Install, configure, and deliver the system to users.	Released software product

Key Characteristics of OO Development Cycle:

- Iterative: The cycle repeats for each release/increment with refinements.
- Incremental: Each iteration adds new functionality to the existing system.
- Use-case driven: Requirements are captured as use cases which guide all phases.
- Architecture-centric: A stable software architecture is established early.
- Risk-reduction: Early iterations address high-risk areas first.

Q2. Define Object-Oriented Analysis (OOA) and Object-Oriented Design (OOD). Explain Software Development Life Cycle (SDLC) in the context of OOAD. [2+5] (2081 Chaitra)

Answer:

Object-Oriented Analysis (OOA)

OOA is the process of analyzing a problem domain to identify the real-world objects (entities), their properties (attributes), behaviors (operations), and relationships. It answers the question: WHAT must the system do? The key deliverable of OOA is a Conceptual Model / Domain Model.

Object-Oriented Design (OOD)

OOD is the process of transforming the conceptual model from OOA into a detailed software solution — defining software classes, their attributes, methods, and how objects interact via messages. It answers: HOW will the system do it? Key deliverable: Design Class Diagram and Interaction Diagrams.

SDLC in the Context of OOAD

SDLC Phase	OOAD Activity	Key Artifacts
Requirements	Identify actors, use cases, functional & non-functional requirements	Use Case Diagram, SRS Document
Analysis (OOA)	Build domain model, identify conceptual classes, create SSDs	Domain Model, CRC Cards
Design (OOD)	Create design classes, apply GRASP patterns, build interaction diagrams	Design Class Diagram, Sequence Diagram
Implementation (OOP)	Code classes and methods from design diagrams	Source Code (Java/C++)
Testing	Verify correctness against use cases and requirements	Test Cases, Bug Reports
Maintenance	Modify and enhance system in future iterations	Updated models and code

---

Q3. Explain the OO Development Cycle compared with the classical approach. [5] (2081 Ashwin)

Answer:

Aspect	Classical (Waterfall) Approach	OO Development Cycle
Structure	Linear, sequential phases	Iterative, incremental cycles
Flexibility	Rigid — changes are costly after a phase	Flexible — requirements can evolve
Delivery	Single delivery at the end	Multiple partial deliveries per iteration

Risk	High risk — problems detected late	Low risk — early iterations reduce risk
Modeling Unit	Functions and procedures	Objects and classes
Reuse	Limited code reuse	High reuse via inheritance & polymorphism
User Feedback	Late — only after full system delivered	Early — feedback at each iteration
Testing	Done at end (testing phase)	Done continuously in each iteration
Suitable For	Stable, well-understood requirements	Complex, evolving requirements

Advantages of OO Development Cycle over Classical Approach:

- Accommodates changing requirements without starting over.
- Produces working software earlier through incremental delivery.
- Promotes reusability through encapsulation, inheritance, and polymorphism.
- Better manages complexity in large systems through modularity.

SECTION 2: Fundamental Concepts of Object Orientation

Exam Ref: 2082 Kartik, 2080 Ashwin [5+3], 2079 Chaitra, 2078 Poush [8], 2077 Chaitra [4+6], 2075 Baisakh [6], 2074 Bhadra [4+6]

Q4. Explain and illustrate the fundamental concepts of object orientation. What is the key difference between conventional design approaches to modern OO design cycle? [4+6] (2077 Chaitra)

Answer:

Fundamental Concepts of Object Orientation

1. Object

An object is any real-world entity that has a state (attributes/data) and behavior (methods/operations). An object is an instance of a class. Example: A 'Car' object has attributes: color, speed, model; and behaviors: accelerate(), brake(), honk().

2. Class

A class is a blueprint or template that defines the attributes and behaviors common to all objects of a particular type. Example: Class 'Student' defines attributes: studentID, name, GPA and methods: enroll(), getGrade().

3. Encapsulation

Encapsulation is the mechanism of bundling data (attributes) and methods (operations) together within a class, and hiding internal implementation details from the outside world. It is achieved using access modifiers: private (–), protected (#), public (+). Example: A BankAccount class hides its balance and only allows access via deposit() and withdraw() methods.

4. Inheritance

Inheritance is the mechanism by which one class (child/subclass) inherits the attributes and methods of another class (parent/superclass). It promotes code reuse and models 'is-a' relationships. Example: 'SavingsAccount' inherits from 'BankAccount'.

5. Polymorphism

Polymorphism means 'many forms'. It allows the same method name to behave differently in different classes. Types: (a) Compile-time (method overloading), (b) Run-time (method overriding). Example: draw() method behaves differently in Circle, Rectangle, and Triangle classes.

6. Abstraction

Abstraction means showing only essential features and hiding unnecessary complexity. Example: A Driver uses a Car without knowing the internal engine mechanics.

7. Message Passing

Objects communicate by sending messages to each other — calling each other's methods. Example: order.calculateTotal() sends a message to the order object.

Concept	Key Idea	Benefit
Object	Instance of a class with state and behavior	Models real-world entities naturally
Class	Blueprint for creating objects	Promotes consistency and reuse
Encapsulation	Data hiding + bundling	Security, modularity, easy maintenance
Inheritance	'Is-a' relationship, code reuse	Reduces redundancy, promotes hierarchy
Polymorphism	Same name, different behavior	Flexibility and extensibility
Abstraction	Hide complexity, show essentials	Simplifies programming

Q5. Explain the principles of object oriented approach. Discuss the benefits of OO approach over the traditional functional approach. [5+3] (2080 Ashwin Back)

Answer:

Principles of OO Approach

- Single Responsibility Principle: Each class should have only one reason to change — one responsibility.
- Open/Closed Principle: Classes should be open for extension but closed for modification.
- Liskov Substitution Principle: Subclass objects should be substitutable for superclass objects.
- Interface Segregation: Clients should not be forced to depend on interfaces they do not use.
- Dependency Inversion: Depend on abstractions, not on concrete implementations.

OO Approach vs. Traditional Functional Approach

Aspect	Traditional/Functional Approach	Object-Oriented Approach
Basic Unit	Functions / Procedures	Objects / Classes
Data & Behavior	Separated (data and functions apart)	Bundled together (encapsulation)
Code Reuse	Limited — functions tightly coupled	High — through inheritance, polymorphism
Data Security	Global data accessible anywhere	Data hidden via access modifiers
Modeling	Abstract from algorithms (HOW)	Models real world (WHAT)
Maintenance	Difficult — changes affect many functions	Easy — changes confined to class
Example Language	C, Pascal, FORTRAN	Java, C++, Python, C#

SECTION 3: Requirements Process — Functional & Non-Functional Requirements

Exam Ref: 2081 Ashwin Q8d, 2080 Chaitra Q1, 2079 Chaitra Q3, 2075 Baisakh Q1b, 2072 Ashwin Q1a

Q6. What is requirement gathering? Differentiate between functional and non-functional requirement. [1+5] (2080 Chaitra)

Answer:

Requirement Gathering

Requirement gathering is the process of systematically collecting, documenting, and validating the needs and expectations of stakeholders for a software system. It is the first and most critical phase of software development.

Methods of Requirement Gathering:

- Interviews: Direct conversation with stakeholders.
- Questionnaires: Structured forms distributed to users.
- Observation: Watching users perform tasks in their environment.
- Workshops / JAD Sessions: Group sessions with all stakeholders.
- Prototyping: Building a rough model to validate understanding.

Functional vs. Non-Functional Requirements

Aspect	Functional Requirements	Non-Functional Requirements
Definition	Define WHAT the system should do — specific behaviors, features, and functions.	Define HOW WELL the system performs — quality attributes and constraints.
Focus	System behavior and functionality	System quality, performance, constraints
Measurability	Tested by checking if the function works	Measured using metrics (response time, uptime%)
Also Called	Behavioral requirements	Quality-of-service / constraints / NFRs
Examples	• User can log in with username & password • System shall generate invoice • User can search for books by ISBN	• System shall respond in < 2 seconds • System uptime shall be 99.9% • Password must be encrypted with AES-256
Testing	Functional testing, UAT	Performance, load, security, compliance testing

Types of Non-Functional Requirements (NFRs):

- Performance: Speed, throughput, response time.
- Reliability: System must work correctly over time.
- Scalability: Ability to handle growing workload.
- Security: Protection against unauthorized access.
- Usability: Ease of use for end users.
- Maintainability: Ease of modifying and upgrading the system.

Q7. Briefly explain the requirement elicitation process in OOA. List different types of requirement used to describe the object oriented system. [3+2] (2078 Chaitra)

Answer:

Requirement Elicitation Process in OOA

- Step 1 — Identify Stakeholders: Determine who will use or be affected by the system.
- Step 2 — Understand the Domain: Study the problem area using interviews and documents.

- Step 3 — Discover Requirements: Elicit functional and non-functional requirements.
- Step 4 — Document Requirements: Capture requirements as use cases or SRS.
- Step 5 — Validate Requirements: Review with stakeholders for completeness and correctness.
- Step 6 — Manage Requirements: Track changes to requirements throughout the project.

Types of Requirements in OO Systems:

- Functional Requirements: What the system should do (features, behaviors).
- Non-Functional Requirements: Quality attributes — performance, security, reliability.
- Domain Requirements: Requirements derived from the application domain.
- User Requirements: High-level needs stated by end users.
- System Requirements: Detailed technical specifications for developers.
- Interface Requirements: How the system interacts with users and other systems.

SECTION 4: Use Cases — Include, Extend Relationships & Diagrams

Exam Ref: 2082 Kartik Q4 [2+6], 2081 Chaitra Q3 [2+6], 2080 Ashwin Q4 [2+6+6+6], 2079 Chaitra, 2078 Chaitra Q2, 2078 Baishakh Q5a

Q8. Differentiate between include and extend relationship. Draw a Use Case Diagram for an Online Job Portal with actors like Job Seeker, Employer, and Admin. [2+6] (2082 Kartik)

Answer:

Use Case Relationships — Include vs. Extend

Aspect	<<include>> Relationship	<<extend>> Relationship
Definition	A use case ALWAYS includes the behavior of another — it is mandatory.	A use case OPTIONALLY extends the behavior of another — it is conditional.
Direction	Arrow points FROM base TO included use case.	Arrow points FROM extension TO base use case.
Optionality	MANDATORY — base always calls the included	OPTIONAL — only when condition met
Purpose	Avoids duplication of common sub-flows.	Adds optional/exceptional behavior without modifying base.
Example	Login <<include>> Validate Credentials (Login always validates)	Withdraw Cash <<extend>> Print Receipt (Receipt only if user requests)

Use Case Diagram for Online Job Portal — Actors & Use Cases:

Actor	Use Cases
Job Seeker	Register Account, Login (<<include>> Validate Credentials), Search Jobs, Apply for Job (<<include>> Upload Resume), View Application Status, Update Profile
Employer	Register Company, Login (<<include>> Validate Credentials), Post Job, View Applications, Shortlist Candidates, Send Interview Invite (<<extend>> View Applications)
Admin	Login (<<include>> Validate Credentials), Manage Users, Approve Job Postings, Generate Reports, Manage Categories

Key Relationships:

- 'Apply for Job' <<include>> 'Upload Resume' — resume upload is mandatory.
- 'Login' <<include>> 'Validate Credentials' — every login must validate.
- 'Send Interview Invite' <<extend>> 'View Applications' — only when shortlisted.

Q9. Differentiate between include and extend relationship. Identify actors, use-cases and relationships from the scenario (Ministry of Health and Population system) and draw the use case diagram. [2+6] (2081 Chaitra)

Answer:

Actors Identified:

- Administrator: Has full access — logs in, enters and modifies data.
- Normal User (Public): Can view data, visualize data, view forecasted data.
- System (Timer): Updates data monthly — background actor.

Use Cases Identified:

Actor	Use Case

Public/User	View Home Page, View Population Data (by zone/district/ward), Visualize Data (graphical/map/tabular), View Forecasted Data
Administrator	Login (<<include>> Validate Credentials), Enter Data, Modify Data, View Population Data
System	Update Data Monthly (scheduled trigger)

Relationships:

- 'Login' <<include>> 'Validate Credentials' — Admin must always validate before access.
- 'Enter Data' and 'Modify Data' are exclusive to Administrator.
- 'Visualize Data' <<extend>> 'View Population Data' — visualization is optional.
- 'View Forecasted Data' <<extend>> 'View Population Data' — optional extension for users.

Q10. Explain different types of actors in Use Case diagram. Draw Use Case diagram, Activity diagram and Sequence Diagram for the Boarding Pass / Check-In scenario. [2+6+6+6] (2080 Chaitra)

Answer:

Types of Actors in Use Case Diagram

Type	Description	Example
Primary Actor	Initiates the use case to achieve a goal. Directly benefits from the use case.	Passenger, Clerk in the scenario
Secondary Actor	Provides services required by the system but does not initiate.	Airlines Staff (wheelchair), Database
External System Actor	Another software system that interacts with this system.	Payment Gateway, Flight Database
Time Actor	System itself triggers the use case at a specific time.	Monthly report generation

Actors in Boarding Pass Scenario:

- Passenger (Primary) — checks in, requests boarding pass.
- Minor Passenger — special case (no ticket needed).
- Clerk (Primary) — issues boarding pass, verifies documents.
- Airlines Staff (Secondary) — provides wheelchair service.

Use Cases:

- Issue Boarding Pass (<<include>> Verify ID, <<include>> Verify Ticket)
- Verify Ticket — <<extend>> with 'Skip for Minor' condition.
- Weigh Baggage — <<extend>> 'Charge Extra Baggage Fee' if overweight.
- Check-In with Boarding Pass
- Request Wheelchair Facility — <<extend>> Check-In if passenger has disability.

SECTION 5: Unified Process (UP) / Rational Unified Process (RUP)

Exam Ref: 2082 Q10a, 2081 Ashwin Q8d, 2080 Ashwin Q8a, 2079 Ashwin Q1 [4+4], 2078 Baishakh Q1 [2+4], 2076 Baisakh Q1 [6]

Q11. What are the differences of OO Development Cycle to other conventional approach? Explain different phases of Rational Unified Process. [4+4] (2079 Ashwin Back)

Answer:

Rational Unified Process (RUP) / Unified Process (UP)

The Unified Process (UP) is an iterative software development framework that is Use-Case Driven, Architecture-Centric, and Iterative & Incremental. It organizes development into 4 phases:

Phase	Primary Goal	Key Activities	Milestone
1. Inception	Define scope, feasibility, business case.	Identify stakeholders, high-level use cases, risk assessment.	LCO — Go/No-Go decision
2. Elaboration	Mitigate major risks, establish architecture.	Refine use cases, build architectural prototype, domain model.	LCA — Architecture stable
3. Construction	Build the system incrementally.	Detailed design, coding, testing, integration, beta testing.	IOC — System ready for beta
4. Transition	Deploy system to users, fix issues.	User training, deployment, bug fixing, performance tuning.	Product Release — System live

RUP Disciplines (run across all phases):

- Business Modeling: Understand the business domain.
- Requirements: Capture what the system must do (use cases).
- Analysis & Design: Transform requirements into design.
- Implementation: Code the design.
- Test: Verify and validate the system.
- Deployment: Deliver the system to users.

Q12. Define concept of OOAD. Why is it so important? Explain different phases of Unified Process (UP). [2+4] (2078 Baishakh)

Answer:

OOAD — Definition and Importance

Object-Oriented Analysis and Design (OOAD) is a software engineering approach that models a system as a group of interacting objects. It uses OO concepts (class, object, inheritance, encapsulation, polymorphism) throughout all phases of software development.

Why OOAD is Important:

- Natural Mapping: OO models map closely to real-world entities.
- Modularity: Systems are built as independent objects — easy to develop and maintain.
- Reusability: Inheritance and polymorphism enable reuse of existing classes.
- Scalability: OO systems scale more easily than procedural systems.
- Reduces Complexity: Abstraction and encapsulation hide complexity.
- Bridges Gap: Reduces the semantic gap between problem domain and software solution.

SECTION 6: UML — Fundamentals, Building Blocks, Views & Notations

Exam Ref: 2081 Chaitra Q2 [2+6], 2077 Chaitra Q2 [2+6], 2079 Jestha Q1 [2+2], 2072 Magh Q1a [6]

Q13. Why do we need UML in software development? Explain dependency, aggregation and composition with proper symbols. [2+6] (2081 Chaitra)

Answer:

Need for UML in Software Development

- Standardization: Provides a common visual language understood by all developers worldwide.
- Communication: Enables clear communication between developers, clients, and stakeholders.
- Documentation: Creates lasting documentation of system structure and behavior.
- Visualization: Makes abstract software concepts visible through diagrams.
- Blueprint for Code: Design diagrams translate directly into class skeletons in OO languages.
- Analysis Tool: Helps identify design flaws early before expensive coding begins.

UML Relationships — Dependency, Aggregation, Composition, and Generalization

Relationship	Symbol/Notation	Meaning	Example
Dependency	Dashed arrow: A - - - > B	Class A uses/depends on class B. Change in B may affect A. Weakest relationship.	OrderReport depends on Order class.
Association	Solid line: A ——— B	Structural link between two classes. Objects of A know about objects of B.	Student is associated with Course.
Aggregation	Hollow diamond at whole end: A <>—— B	'Has-a' (weak). Part can exist independently of whole.	Department <>—— Professor (Professor exists without department)
Composition	Filled diamond at whole end: A *—— B	'Has-a' (strong). Part cannot exist without whole. Destroying whole destroys parts.	House *—— Room (Room ceases without House)
Generalization	Solid line with hollow triangle: A ——> B	Inheritance / 'Is-a' relationship. Child inherits from parent.	SavingsAccount ——> BankAccount

Key Difference — Aggregation vs. Composition:

- Aggregation: Weak 'has-a'. Parts can exist independently. Example: Library <>—— Book (books exist outside the library).
- Composition: Strong 'has-a'. Parts cannot exist without whole. Example: Order *—— OrderLine (order line meaningless without an order).

Q14. Describe about Unified Modeling Language and list out to which models the following diagrams belong. [2+2] (2079 Jestha)

Answer:

Diagram	Category / Model	Purpose
Use Case Diagram	Behavioral / Requirements Model	Shows system functionality from actor's perspective.
Activity Diagram	Behavioral / Dynamic Model	Shows workflow, process flow, and control flow.

Conceptual Model / Domain Model	Analysis Model	Identifies real-world conceptual classes in the problem domain.
Sequence Diagram	Interaction Model (Behavioral)	Shows time-ordered message exchanges between objects.
Collaboration Diagram	Interaction Model (Behavioral)	Shows structural organization of objects that send and receive messages.
State Diagram	Behavioral / Dynamic Model	Shows states of an object and transitions between states.
Class Diagram	Structural Model	Shows classes, attributes, methods, and their static relationships.

Q15. Why do we need different types of system models? What are the building blocks of UML? Explain in brief. [2+6] (2077 Chaitra)

Answer:

Why Different System Models are Needed

- No single diagram can capture all aspects of a complex system.
- Structural models show WHAT the system is made of (classes, components).
- Behavioral models show WHAT the system does (activity, state).
- Interaction models show HOW objects collaborate (sequence, collaboration).
- Different stakeholders need different views — clients need use case models, developers need class diagrams.

Building Blocks of UML — Three Categories:

Category	Type	Description	Examples
1. Things	Structural	Static parts of the model.	Class, Interface, Component, Node, Use Case
1. Things	Behavioral	Dynamic parts representing behavior.	Interaction, State Machine, Activity
1. Things	Grouping	Organizational elements.	Package
1. Things	Annotational	Explanatory notes.	Note/Comment
2. Relationships	Dependency	Change in one affects another.	Dashed arrow (- - ->)
2. Relationships	Association	Structural link between classes.	Solid line
2. Relationships	Generalization	Inheritance relationship.	Solid line + hollow triangle
2. Relationships	Realization	Class implements an interface.	Dashed + hollow triangle
3. Diagrams	Structural (7)	Show static architecture.	Class, Component, Object, Package...
3. Diagrams	Behavioral (7)	Show system dynamics.	Use Case, Activity, State, Sequence...

SECTION 7: Defining Models — Conceptual, Structural & Behavioral Models

Exam Ref: 2079 Chaitra Q3, 2075 Baisakh, 2074 Bhadra Q2, 2073 Magh Q2a, 2072 Magh Q2, 2078 Chaitra

Q16. What are the different views of OOA? [3] (2079 Chaitra)

Answer:

Model	Also Called	Focus	Diagrams Used
Structural Model	Static / Class Model	Classes, attributes, methods, and relationships — the skeleton.	Class Diagram, Domain Model, Object Diagram
Behavioral Model	Dynamic Model	How objects change state over time.	State Diagram, Activity Diagram
Interaction Model	Functional / Use Case Model	How objects collaborate and communicate.	Use Case Diagram, Sequence Diagram, Collaboration Diagram, SSD

Q17. In OOAD, there are various types of models — conceptual, structural, behavioral, etc. What is the significance of these many different types of model? Explain with illustrative examples. [7] (2079 Ashwin Back, 2080 Ashwin Back)

Answer:

Significance of Different Types of Models in OOAD

- Complexity: No single diagram can capture all aspects of a real system.
- Multiple Stakeholders: Clients understand use case models; developers need class diagrams.
- Different Aspects: Structure, behavior, and interaction are all important.
- Verification: Cross-checking across models catches inconsistencies early.

Model Type	Purpose	Diagram	Example (Library System)
Conceptual Model	Identifies real-world concepts in the problem domain. Used in OOA.	Domain Class Diagram (no methods)	Classes: Book, Member, Loan, Librarian. Member 'borrows' Book.
Structural Model	Shows the static architecture — classes, attributes, methods.	Design Class Diagram	Book class: -title:String, -ISBN:String, +search():Book
Behavioral Model	Shows how objects change state in response to events.	State / Activity Diagram	Loan State: Requested → Approved → Issued → Returned → Overdue
Interaction Model	Shows how objects collaborate to implement functions.	Sequence / Collaboration Diagram	Member requests book → Check availability → Issue Loan → Update Inventory
Use Case Model	Captures system functionality from user's perspective.	Use Case Diagram	Actors: Member, Librarian. Use cases: Borrow Book, Return Book

SECTION 8: System Behavior, SSD & Incremental Development

Exam Ref: 2079 Jestha Q2, 2078 Baishakh Q4, 2075 Baisakh Q2, 2072 Magh Q2a

Q18. Describe incremental and iterative development process with suitable example. [4] (2079 Jestha)

Answer:

Iterative Development

Iterative development means the project is divided into a series of iterations (fixed-length mini-projects, typically 2–4 weeks). At the end of each iteration, a tested, integrated, and partially complete system is produced. Each iteration refines and expands the previous one.

Incremental Development

Incremental development means each iteration adds a new increment (feature/subset) to the working system. The system 'grows' increment by increment until the full system is delivered.

Example — Online Library System:

Iteration	Increment Added	What is Delivered
Iteration 1 (2 weeks)	Login and User Authentication	Working login page, user registration, password validation.
Iteration 2 (2 weeks)	Book Search and Catalog Browse	Full book search, category browse, book details page.
Iteration 3 (2 weeks)	Borrow and Return Book	Issue book, return book, due date tracking, fine calculation.
Iteration 4 (2 weeks)	Librarian Admin Functions	Add/remove books, manage members, generate reports.
Iteration 5 (2 weeks)	Notifications and Final Polish	Email reminders, overdue alerts, UI improvements, deployment.

Benefits of Iterative and Incremental Development:

- Early delivery of working software increases customer confidence.
- Risks are identified and mitigated early.
- Requirements can be refined based on user feedback after each iteration.
- Testing is continuous — integration bugs are caught early.

Q19. How is a system behavior represented in OOA? How do we construct a System Sequence Diagram (SSD) from a Use Case? Explain with a SSD for a process Sale scenario. [8] (2078 Baishakh)

Answer:

System Behavior in OOA

System behavior in OOA describes how the system responds to external events initiated by actors. It is captured using:

- Use Cases: Textual description of interactions between actors and the system.
- System Sequence Diagrams (SSD): Graphical representation of the sequence of events between actors and the system during one use case scenario.
- System Operations: Operations the system must support (identified from SSDs).
- Operation Contracts: Pre and post-conditions of system operations.

Steps to Construct an SSD:

- Step 1: Select a use case scenario (main success scenario).
- Step 2: Identify the primary actor who initiates the use case.
- Step 3: Show actor as stick figure on left, System as box on right, with lifelines.

- Step 4: Identify system events (actor → system) and show as solid arrows.
- Step 5: Identify system responses (system → actor) and show as dashed arrows.
- Step 6: Add loops, conditions (guard conditions in [ ]), and alternative flows.

SSD for 'Process Sale' Use Case (POS Terminal):

Step	Event / Response	Direction
1	makeNewSale()	Cashier → System
2	Display: new sale started	System → Cashier
3 (loop)	enterItem(itemID, qty) — for each item	Cashier → System
4	Description, price, running total displayed	System → Cashier
5	endSale()	Cashier → System
6	Total with taxes calculated and displayed	System → Cashier
7	makePayment(amount)	Cashier → System
8	Change due displayed, receipt printed	System → Cashier

QUICK REVISION: Chapter 1 — Key Points Summary

Topic	Key Points	Expected Marks
OOP / OOA / OOD	OOA=WHAT, OOD=HOW, OOP=CODE. All share same OO concepts.	3–5
OO Development Cycle	Iterative + Incremental. Phases: Planning→OOA→OOD→OOP→Testing→Deployment.	4–6
OO vs Classical	OO: iterative, object-based, reusable, risk-reducing. Classical: linear, function-based, rigid.	4–5
OO Concepts	Encapsulation, Inheritance, Polymorphism, Abstraction, Object, Class, Message Passing.	4–6
Requirements	Functional = WHAT (behavior). Non-Functional = HOW WELL (quality, performance).	3–5
Use Cases	Include = mandatory sub-flow. Extend = optional/conditional extension.	4–6
RUP/UP Phases	Inception→Elaboration→Construction→Transition. Iterative, use-case driven.	4–6
UML Need	Standardization, visualization, communication, documentation, code blueprint.	2–4
UML Relationships	Dependency (weakest) → Association → Aggregation (weak has-a) → Composition (strong has-a) → Generalization.	4–6
UML Building Blocks	Things (structural/behavioral/grouping/annotational) + Relationships + Diagrams.	4–6
UML Diagram Types	Use Case, Class, Sequence, Activity, State diagrams.	2–4
System Models	Structural (WHAT), Behavioral (STATE), Interaction (WHO/HOW).	4–6
SSD	Actor-system interactions in time order for a use case scenario.	4–6
Iterative & Incremental	Each iteration adds a tested increment. Early feedback, continuous testing.	3–4